



Laporan Analisis Program

Nama : **Faizal Rahman**
NIM : **20230040262**
Kelas : **TI24E**
Semester : **4 (Empat)**

A. Deskripsi Program

1. Nama Aplikasi

Aplikasi yang dikembangkan adalah "Sistem Klinik Sederhana". Program ini merupakan aplikasi desktop berbasis Java yang dirancang untuk membantu pengelolaan administrasi klinik secara lebih terstruktur dan efisien. Aplikasi ini menggunakan Java Swing sebagai antarmuka pengguna dan MySQL sebagai media penyimpanan data.

2. Tujuan Aplikasi

Tujuan utama dari aplikasi ini adalah mempermudah proses pengelolaan data pasien, data dokter, serta data janji temu di klinik. Selain itu, aplikasi ini juga bertujuan untuk mengurangi kesalahan pencatatan manual, mempercepat pencarian data, dan membantu pihak klinik dalam membuat laporan sederhana mengenai jumlah pasien, dokter, dan transaksi yang terjadi. Aplikasi ini juga dirancang agar pengguna dapat masuk ke sistem melalui fitur login sebelum mengakses menu utama.

3. Permasalahan yang Diselesaikan

Sebelum adanya aplikasi ini, pengelolaan data klinik sering dilakukan secara manual sehingga rentan terhadap kesalahan pencatatan, duplikasi data, serta kesulitan dalam mencari informasi dengan cepat dan akurat. Program ini membantu mengatasi masalah tersebut dengan menyediakan sistem pencatatan yang terorganisasi, penyimpanan data yang lebih aman di database, serta fasilitas pencarian dan pelaporan yang lebih praktis. Dengan demikian, kegiatan administrasi klinik menjadi lebih cepat, rapi, dan terkontrol.

B. Analisis Konsep PBO

1. Class dan Object

- Lokasi class atau kode yang digunakan:
 - Person.java
 - User.java
 - Patient.java
 - Doctor.java
 - Appointment.java
- Alasan penggunaan konsep tersebut:
Setiap entitas penting dalam sistem klinik, seperti pasien, dokter, pengguna, dan janji temu, dibuat sebagai class. Hal ini karena setiap entitas memiliki data dan perilaku yang berbeda, sehingga perlu dikemas dalam satu unit yang terstruktur.
- Manfaatnya terhadap aplikasi:
 - Program menjadi lebih rapi dan mudah dipahami.
 - Data dan perilaku setiap entitas dapat dikelola secara terpisah.
 - Memudahkan pengembangan fitur baru di masa depan.

- Potongan kode:
 - Contoh class Patient pada Patient.java

```
src > model > Patient.java > Patient
1 package model;
2
3 public class Patient extends Person {
4     private String birthDate;
5     private String gender;
6     private String medicalHistory;
7
8     public Patient() {
9     }
10
11     public Patient(int id, String fullName, String phone, String address, String birthDate, St:
12         super(id, fullName, phone, address);
13         this.birthDate = birthDate;
14         this.gender = gender;
15         this.medicalHistory = medicalHistory;
16     }
17 }
```

- Contoh object digunakan pada form PatientForm.java

```
src > ui > PatientForm.java > {} ui
1 package ui;
2
3 import model.Patient;
4 import service.ClinicService;
5
6 import javax.swing.*;
7 import java.awt.*;
8 import java.sql.SQLException;
9 import java.util.List;
10
11 public class PatientForm extends JDialog {
12     private final ClinicService clinicService = new ClinicService();
13     private final DefaultListModel<String> listModel = new DefaultListModel<>();
14     private final JList<String> patientList = new JList<>(listModel);
15     private final JTextField txtName = new JTextField();
16     private final JTextField txtPhone = new JTextField();
17     private final JTextField txtAddress = new JTextField();
18     private final JTextField txtBirthDate = new JTextField();
19     private final JTextField txtGender = new JTextField();
20     private final JTextArea txtHistory = new JTextArea();
21     private int selectedId = 0;
22 }
```

2. Inheritance

- Lokasi class atau kode yang digunakan:
 - Person.java
 - User.java
 - Patient.java
 - Doctor.java
- Alasan penggunaan konsep tersebut:

Kelas User, Patient, dan Doctor memiliki atribut dasar yang sama, seperti nama, telepon, alamat, dan identitas. Karena itu, atribut tersebut diwariskan dari kelas Person agar tidak ditulis ulang berkali-kali.
- Manfaatnya terhadap aplikasi:
 - Mengurangi duplikasi kode.
 - Mempermudah pemeliharaan karena perubahan atribut umum hanya dilakukan di satu tempat.
 - Membuat struktur program lebih terorganisir.

- Potongan kode:
 - Kelas Person sebagai parent class dan User/Patient/Doctor sebagai child class.

```
src > model > Person.java > Person
1 package model;
2
3 public abstract class Person {
4     protected int id;
5     protected String fullName;
6     protected String phone;
7     protected String address;
8
9     public Person() {
10    }
11
12    public Person(int id, String fullName, String phone, String address) {
13        this.id = id;
14        this.fullName = fullName;
15        this.phone = phone;
16        this.address = address;
17    }
18 }
```

```
src > model > User.java > User
1 package model;
2
3 public class User extends Person {
4     private String username;
5     private String password;
6     private String role;
7
8     public User() {
9    }
10 }
```

```
src > model > Patient.java > Patient
1 package model;
2
3 public class Patient extends Person {
4     private String birthDate;
5     private String gender;
6     private String medicalHistory;
7
8     public Patient() {
9    }
10 }
```

```
src > model > Doctor.java > Doctor
1 package model;
2
3 public class Doctor extends Person {
4     private String specialty;
5
6     public Doctor() {
7    }
8
9     public Doctor(int id, String fullName, String phone, String address, String specialty) {
10         super(id, fullName, phone, address);
11         this.specialty = specialty;
12     }
13 }
```

3. Polymorphism

- Lokasi class atau kode yang digunakan:
 - Person.java
 - User.java
 - Patient.java
 - Doctor.java
- Alasan penggunaan konsep tersebut:
Method displayInfo() dideklarasikan dalam kelas Person dan diimplementasikan kembali oleh kelas turunannya. Hal ini menunjukkan bahwa objek dari kelas berbeda dapat memiliki perilaku yang berbeda meskipun dipanggil dengan nama method yang sama.

- Manfaatnya terhadap aplikasi:
 - Program menjadi lebih fleksibel.
 - Memudahkan pemanggilan fungsi yang sama untuk objek berbeda.
 - Meningkatkan efisiensi desain program.
- Potongan kode:
 - `displayInfo()` pada `Person.java`

```
src > model > Person.java > Person > displayInfo()
3 public abstract class Person {
18
19 public abstract void displayInfo();
20
21 public int getId() {
22     return id;
23 }

src > model > Doctor.java > Doctor > displayInfo()
3 public class Doctor extends Person {
13
14 @Override
15 public void displayInfo() {
16     System.out.println("Dokter: " + getFullName() + " - " + specialty);
17 }
18
```

4. Encapsulation

- Lokasi class atau kode yang digunakan:
 - `User.java`
 - `Patient.java`
 - `Doctor.java`
 - `Appointment.java`
- Alasan penggunaan konsep tersebut:
Atribut-atribut pada kelas dibuat `private` agar tidak bisa diakses langsung dari luar kelas. Data hanya bisa diakses dan dimodifikasi melalui `getter` dan `setter`.
- Manfaatnya terhadap aplikasi:
 - Menjaga keamanan dan integritas data.
 - Mencegah perubahan data yang tidak disengaja.
 - Membuat kode lebih aman dan terkontrol.
- Potongan kode:
 - Atribut `private String username;` dan `method getUsername()/setUsername()` pada `User.java`

```
src > model > User.java > User > username
3 public class User extends Person {
4     private String username;
5     private String password;
6     private String role;

src > model > User.java > User > getUsername()
3 public class User extends Person {
27
28 public String getUsername() {
29     return username == null ? "" : username;
30 }

src > model > User.java > User > setUsername(String)
3 public class User extends Person {
32
33 public void setUsername(String username) {
34     this.username = username == null ? "" : username.trim().toLowerCase();
35 }
```

5. Abstract Class atau Interface

- Lokasi class atau kode yang digunakan:
 - Person.java
- Alasan penggunaan konsep tersebut:
Person dibuat sebagai abstract class karena kelas ini berfungsi sebagai kerangka dasar untuk entitas yang memiliki sifat umum, tetapi tidak perlu diinstansiasi langsung.
- Manfaatnya terhadap aplikasi:
 - Memberikan struktur dasar yang konsisten untuk semua entitas.
 - Memaksa subclass untuk mengimplementasikan behavior yang penting.
 - Membuat desain program lebih terarah dan mudah dikembangkan.
- Potongan kode:
 - public abstract class Person pada Person.java

```
src > model > Person.java > Person
1 package model;
2
3 public abstract class Person {
4     protected int id;
5     protected String fullName;
6     protected String phone;
7     protected String address;
8 }
```

6. Package

- Lokasi class atau kode yang digunakan:
 - model
 - ui
 - service
 - repository
 - database
- Alasan penggunaan konsep tersebut:
Kode dikelompokkan berdasarkan fungsinya agar lebih rapi dan mudah dipahami. Misalnya, semua kelas data disimpan di package model, tampilan di package ui, dan logika bisnis di package service.
- Manfaatnya terhadap aplikasi:
 - Memudahkan pemeliharaan program.
 - Memperjelas pembagian tanggung jawab antar bagian aplikasi.
 - Membantu proses pengembangan dan debugging.
- Potongan kode:
 - package model;, package ui;, package service; pada masing-masing file.

```
src > service > ClinicService.java > ...
1 package service;
2
3 import model.Appointment;
4 import model.Doctor;
5 import model.Patient;
6 import model.User;
7 import repository.ClinicRepository;

src > App.java > ...
1 import database.DatabaseConnection;
2 import ui.LoginFrame;
3
```

7. Collection

- Lokasi class atau kode yang digunakan:
 - ClinicService.java
 - ClinicRepository.java
 - PatientForm.java
 - DoctorForm.java
 - AppointmentForm.java
- Alasan penggunaan konsep tersebut:
Aplikasi perlu menangani banyak data sekaligus, seperti daftar pasien, daftar dokter, dan daftar janji temu. Karena itu, digunakan struktur data collection seperti List.
- Manfaatnya terhadap aplikasi:
 - Memudahkan penyimpanan dan pengolahan data lebih dari satu objek.
 - Menyederhanakan proses pencarian, penambahan, dan penghapusan data.
 - Membuat program lebih dinamis dan efisien.
- Potongan kode:
 - List<Patient>, List<Doctor>, dan List<Appointment> pada ClinicService.java

```
src > service > ClinicService.java > ...
16 public class ClinicService {
47     public List<Patient> getAllPatients() throws SQLException {
48         return repository.getAllPatients();
49     }
}

src > service > ClinicService.java > ...
16 public class ClinicService {
91     public List<Doctor> getAllDoctors() throws SQLException {
92         return repository.getAllDoctors();
93     }
}

src > service > ClinicService.java > ...
16 public class ClinicService {
108     public List<Appointment> getAllAppointments() throws SQLException {
109         return repository.getAllAppointments();
110     }
111
112     public List<Appointment> searchAppointments(String keyword) throws SQLException {
113         if (keyword == null || keyword.trim().isEmpty()) {
114             return getAllAppointments();
115         }
116         return repository.searchAppointments(keyword.trim());
117     }
}
```

8. String Method

- Lokasi class atau kode yang digunakan:
 - User.java
 - ClinicService.java
- Alasan penggunaan konsep tersebut:
 - Program menerima input dari pengguna, seperti username, password, nama, dan alamat. Karena itu, digunakan method string seperti trim(), toLowerCase(), dan equalsIgnoreCase() untuk memastikan data yang masuk valid dan konsisten.
- Manfaatnya terhadap aplikasi:
 - Membantu validasi input.
 - Mengurangi kesalahan akibat spasi atau format teks yang tidak sesuai.
 - Meningkatkan kualitas data yang disimpan.

- Potongan kode:
 - `username.trim().toLowerCase()` pada `User.java`
 - `if (username == null || username.trim().isEmpty())` pada `ClinicService.java`

```
src > model > User.java > User > setUsername(String)
3 public class User extends Person {
32
33     public void setUsername(String username) {
34         this.username = username == null ? "" : username.trim().toLowerCase();
35     }
36 }

src > service > ClinicService.java > ClinicService > login(String, String)
16 public class ClinicService {
19
20     public User login(String username, String password) throws SQLException {
21         if (username == null || username.trim().isEmpty()) {
22             throw new IllegalArgumentException(s: "Username tidak boleh kosong");
23         }
24         if (password == null || password.trim().isEmpty()) {
25             throw new IllegalArgumentException(s: "Password tidak boleh kosong");
26         }
27     }
28 }
```

9. Exception Handling

- Lokasi class atau kode yang digunakan:
 - `ClinicService.java`
 - `LoginFrame.java`
 - `PatientForm.java`
 - `DoctorForm.java`
 - `AppointmentForm.java`
- Alasan penggunaan konsep tersebut:
 - Dalam aplikasi, sering terjadi kesalahan seperti input kosong, data tidak valid, atau gagal koneksi ke database. Karena itu, program perlu menangani error agar tidak langsung crash.
- Manfaatnya terhadap aplikasi:
 - Program lebih stabil.
 - Pengguna menerima pesan kesalahan yang informatif.
 - Proses debugging menjadi lebih mudah.
- Potongan kode:
 - `catch (IllegalArgumentException | SQLException ex)` pada `LoginFrame.java`

```
src > ui > LoginFrame.java > LoginFrame > LoginFrame()
11 ic class LoginFrame extends JFrame {
14     public LoginFrame() {
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37     btnLogin.addActionListener(e -> {
38         try {
39             User user = clinicService.login(txtUser.getText(), new String(txtPass.getPassword()));
40             JOptionPane.showMessageDialog(this, "Selamat datang " + user.getFullName());
41             new MainFrame(user).setVisible(b: true);
42             dispose();
43         } catch (IllegalArgumentException | SQLException ex) {
44             JOptionPane.showMessageDialog(this, ex.getMessage(), title: "Login Gagal", JOptionPane.ERROR_MESSAGE);
45         }
46     });
47
48     add(panel);
49 }
50 }
```

10. File Handling

- Lokasi class atau kode yang digunakan:
 - ClinicService.java
- Alasan penggunaan konsep tersebut:
 - Aplikasi menyediakan fitur pembuatan laporan, sehingga data laporan perlu disimpan dalam bentuk file agar dapat dibuka kembali maupun dicetak.
- Manfaatnya terhadap aplikasi:
 - Laporan dapat disimpan secara permanen.
 - Memudahkan pencatatan dan arsip data.
 - Menambah nilai praktis aplikasi.
- Potongan kode:
 - Files.createDirectories(...) dan Files.writeString(...) pada ClinicService.java

```
src > service > ClinicService.java > ClinicService > exportReportToFile(String)
16 public class ClinicService {
...
126     public Path exportReportToFile(String fileName) throws SQLException, IOException {
127         String report = buildReport();
128         Path path = Path.of(fileName);
129         Files.createDirectories(path.getParent());
130         Files.writeString(path, report);
131         return path;
132     }
}
```

11. Database

- Lokasi class atau kode yang digunakan:
 - DatabaseConnection.java
 - ClinicRepository.java
- Alasan penggunaan konsep tersebut:
 - Data klinik tidak cukup jika hanya disimpan sementara di memori karena data harus tetap tersedia saat aplikasi ditutup dan dibuka kembali. Karena itu, digunakan database MySQL.
- Manfaatnya terhadap aplikasi:
 - Data tersimpan secara permanen.
 - Lebih aman dan terstruktur.
 - Memudahkan pengelolaan data dalam jumlah yang lebih besar.
- Potongan kode / contoh:
 - Koneksi database pada DatabaseConnection.java

```
src > database > DatabaseConnection.java > DatabaseConnection > DATABASE_NAME
1 package database;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6 import java.sql.Statement;
7
8 public class DatabaseConnection {
9     private static final String BASE_URL = "jdbc:mysql://localhost:3306/";
10    private static final String DATABASE_NAME = "db_klinik";
11    private static final String USER = "root";
12    private static final String PASSWORD = "";
13}
```

C. Refleksi Pengembangan

1. Mengapa memilih studi kasus tersebut?

Saya memilih studi kasus sistem klinik karena tema tersebut dekat dengan kehidupan sehari-hari dan cukup relevan untuk diterapkan dalam pemrograman berbasis objek. Studi kasus ini juga cocok untuk mempelajari berbagai konsep OOP, seperti class, inheritance, encapsulation, dan database.

2. Bagian program yang paling sulit dibuat dan alasannya

Bagian yang paling sulit adalah bagian pengelolaan data antar form, terutama saat menghubungkan data pasien, dokter, dan janji temu ke database. Hal ini sulit karena butuh pemahaman yang baik tentang alur program, komunikasi antar kelas, dan penanganan error agar data tidak rusak.

3. Kesalahan (error) yang paling sering ditemui selama pengembangan dan cara mengatasinya

Error yang paling sering muncul adalah error terkait koneksi database dan input data yang tidak valid. Biasanya penyebabnya adalah konfigurasi MySQL yang belum benar atau format data yang salah. Cara mengatasinya adalah memeriksa koneksi database, memastikan nama database dan user/password sesuai, serta menambahkan validasi input sebelum data disimpan.

4. Jika aplikasi ini dikembangkan kembali, tiga fitur yang ingin ditambahkan

Tiga fitur yang ingin ditambahkan adalah:

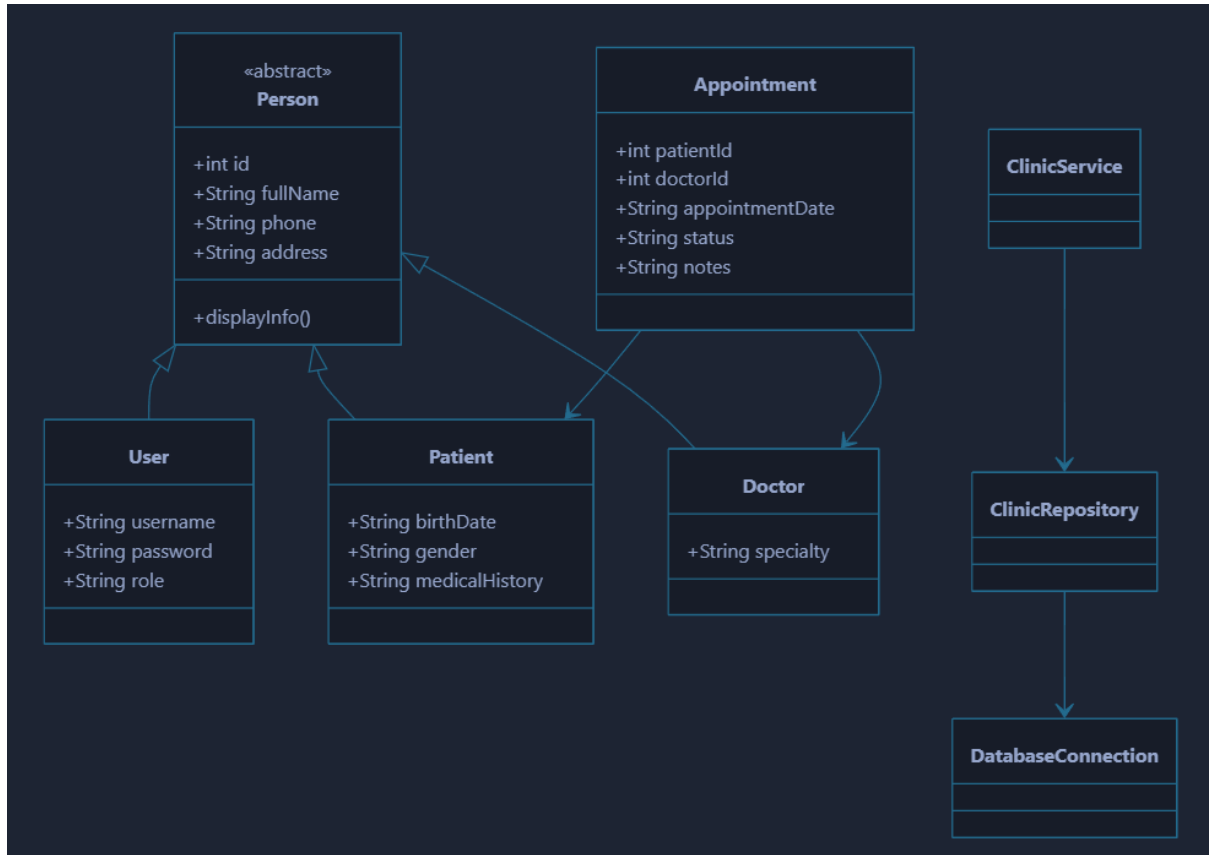
- Fitur pencarian dan filter data yang lebih lengkap
- Fitur cetak laporan atau export laporan ke PDF/Excel
- Fitur notifikasi atau reminder janji temu

5. Konsep PBO yang menurut Anda masih perlu dipelajari lebih lanjut

Konsep yang masih perlu dipelajari lebih lanjut adalah polymorphism dan desain berorientasi objek yang lebih kompleks, terutama dalam pemahaman hubungan antar class dan penerapan prinsip SOLID. Saya juga ingin lebih memahami cara mengelola project dengan struktur class yang lebih rapi dan efisien.

D. Lampiran

1. Class Diagram



Keterangan:

- Person adalah kelas induk abstrak.
- User, Patient, dan Doctor adalah kelas turunan.
- Appointment berhubungan dengan data pasien dan dokter.
- ClinicService dan ClinicRepository berfungsi sebagai penghubung logika bisnis dengan database.

2. Struktur Database

Aplikasi ini menggunakan database MySQL untuk menyimpan data utama. Struktur database yang digunakan terdiri dari beberapa tabel berikut:

a. Tabel users

Tabel ini berisi data akun pengguna yang digunakan untuk login.

Kolom:

- id
- full_name
- username
- password
- role
- phone
- address

b. Tabel patients

Tabel ini berisi data pasien.

Kolom:



- id
- full_name
- phone
- address
- birth_date
- gender
- medical_history

c. Tabel doctors

Tabel ini berisi data dokter.

Kolom:

- id
- full_name
- phone
- address
- specialty

d. Tabel appointments

Tabel ini berisi data janji temu pasien dengan dokter.

Kolom:

- id
- patient_id
- doctor_id
- appointment_date
- status
- notes

Keterangan:

- patient_id dan doctor_id merupakan hubungan antara tabel janji temu dengan tabel pasien dan dokter.

3. Screenshot aplikasi

Username admin

Password

Login

Menu Login



Sistem Klinik Sederhana

Data

Selamat datang Admin Klinik

Role: admin

Menu Utama

Data Pasien

Nama		1 Rayhan 082278876546
Telepon		
Alamat		
Tanggal Lahir		
Gender		
Riwayat		

Menu Data Pasien



Data Dokter

×

Nama

Telepon

Alamat

Spesialis

ID	Nama	Telepon	Alamat	Spesialis
1	Dr. Tirta	085176872343	Jojakarta	Jantung

Simpan

Refresh

Hapus

Menu Data Dokter

Transaksi Janji Temu

×

ID Pasien

ID Dokter

Tanggal

Status

Catatan

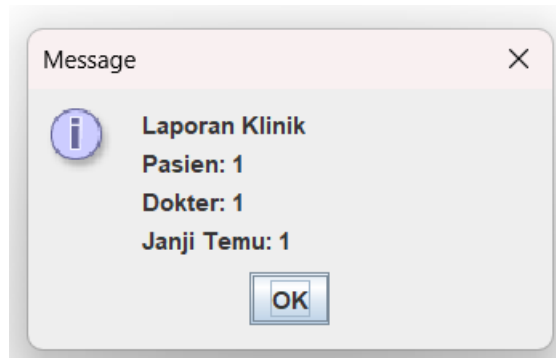
ID	ID Pasien	ID Dokter	Tanggal	Status	Catatan
1	1	1	30/07/2026	Belum	-

Simpan

Refresh

Hapus

Menu Janji Temu



Menu Laporan

4. Penjelasan singkat setiap menu

- a. Login
Menu login berfungsi untuk mengamankan akses aplikasi. Pengguna harus memasukkan username dan password sebelum masuk ke sistem.
- b. Data Pasien
Menu ini digunakan untuk mengelola data pasien, seperti menambah, mencari, dan menghapus data pasien.
- c. Data Dokter
Menu ini digunakan untuk mengelola data dokter, termasuk nama, telepon, alamat, dan spesialis.
- d. Transaksi Janji Temu
Menu ini digunakan untuk mencatat janji temu pasien dengan dokter serta menampilkan informasi tanggal, status, dan catatan.
- e. Laporan
Menu ini berfungsi untuk menampilkan ringkasan data klinik, seperti jumlah pasien, jumlah dokter, dan jumlah janji temu.